PROYECTO XARA

INVESTIGACIÓN Y DESARROLLO

DOCUMENTACIÓN DEL PROYECTO

COORDINACIÓN DE COMPUTADORAS EN RED LOCAL

Integración de técnicas probadas en un solo programa para el usuario general

Proyecto	XARA – Computación distribuida con PCs domésticas
Responsable	LM
Fecha del documento	09/10/2026
Edición	4.2 (ver Anexo B)
Estado	Diseño completado / Prototipo v0.2 verificado en laboratorio

Lima - Perú
2026

TABLA DE CONTENIDO

1. DEFINICIÓN DEL PROYECTO

2. METODOLOGÍA DE TRABAJO

3. HIPÓTESIS, VARIABLES Y MÉTRICAS

4. ESTADO DEL ARTE: CÓMO FUNCIONAN LOS PROYECTOS DE REFERENCIA

5. ALTERNATIVAS Y DECISIÓN DE CONSTRUCCIÓN

6. REQUERIMIENTOS DEL PROTOTIPO

7. ANÁLISIS FUNCIONAL Y DIAGRAMAS

8. ARQUITECTURA DEL PROTOTIPO v0.2

9. PLAN EXPERIMENTAL

10. EVALUACIÓN Y DECISIÓN

11. PLANIFICACIÓN Y CRONOGRAMA

12. RIESGOS

13. REGISTRO AUTOMÁTICO Y EVIDENCIAS

14. MERCADO, CASOS DE USO Y ESTUDIO ECONÓMICO

15. CIERRE Y APROBACIÓN

ANEXOS

1. DEFINICIÓN DEL PROYECTO

1.1 Descripción del problema o necesidad

Muchas personas tienen varias computadoras que, por separado, no alcanzan para ciertas tareas, como ejecutar un modelo de inteligencia artificial local o procesar un volumen grande de trabajo. La memoria de dos equipos no puede sumarse físicamente, pero sí puede repartirse el trabajo entre ellos.

Esto no es una idea por demostrar desde cero. Proyectos como exo, Incredibuild, BOINC, HTCondor, Petals o el modo RPC de llama.cpp ya demuestran que cada parte es posible. El problema es que cada uno resuelve una sola parte, casi todos exigen conocimientos técnicos y varios no funcionan en Windows, que es el sistema del usuario general.

1.2 Propuesta de valor

XARA no inventa una técnica nueva de distribución. Su aporte es **integrar en un solo programa** las técnicas ya probadas, elegir automáticamente cuál conviene en cada caso y **medir si la colaboración realmente mejora** frente a usar un solo equipo. El destinatario es el usuario general con equipos Windows en su casa.

1.3 Objetivo general

Construir y validar un prototipo que coordine computadoras de una misma red local, reparta trabajo entre ellas solo cuando aporte una ventaja medible de capacidad, rendimiento o aprovechamiento, y registre automáticamente la evidencia de cada prueba.

1.4 Objetivos específicos

N.º	Objetivo específico
1	Estudiar cómo funcionan los proyectos de referencia y qué técnica aporta cada uno.
2	Elegir la forma de construir XARA comparando alternativas con criterios explícitos.
3	Implementar un prototipo que descubra equipos, reparta trabajo, recupere fallos y registre datos sin intervención manual.
4	Comparar la ejecución individual frente a la conjunta en cargas independientes y en IA local.
5	Decidir, con umbrales fijados de antemano, si el proyecto continúa, se modifica o se detiene.

1.5 Alcance

Incluye en esta versión	No incluye en esta versión
Cargas independientes por lotes e IA local como casos de prueba	Sumar la RAM para que Windows vea 20 GB
Integración de llama.cpp como motor de IA	Videojuegos o tareas que necesiten respuesta en milisegundos
Pruebas automáticas con registro y veredicto	Ejecutar cualquier programa en otro equipo sin adaptarlo
Pruebas con las dos computadoras disponibles	Compra de hardware, uso de la nube o producto comercial

1.6 Recursos disponibles

Recurso	PC de escritorio	Laptop
Procesador	Intel Core i5-3570 (4 núcleos, sin AVX2)	AMD Ryzen 5 2500U (4 núcleos / 8 hilos, con AVX2)
Memoria RAM	12 GB DDR3	8 GB DDR4
Gráfica	NVIDIA GT 1030, 2 GB	Radeon Vega 8 integrada (~1 GB compartido)
Almacenamiento	SSD 125 GB + HDD 1 TB	SSD 1 TB
Sistema y red	Windows 11 · Ethernet	Windows 11 · Wi-Fi

1.7 Criterios de éxito

- Los equipos se encuentran y se comunican solos, y ninguna tarea se pierde si uno se desconecta.
- Existe al menos una carga en la que la colaboración alcanza el umbral favorable de rendimiento o de capacidad.
- El coste de comunicación no elimina el beneficio obtenido.
- Cada resultado queda respaldado por datos registrados automáticamente y repetibles.
- Una persona sin conocimientos técnicos puede montar y usar el prototipo siguiendo la guía.

2. METODOLOGÍA DE TRABAJO

2.1 Metodología seleccionada

Decisión	Valor
Enfoque	Investigación experimental por etapas, con decisión documentada al cerrar cada etapa
Punto de partida	Técnicas ya demostradas por otros proyectos; se estudian antes de construir
Forma de construir	Capa integradora propia que usa motores existentes (capítulo 5)
Principio de comparación	Ninguna configuración conjunta es válida sin su referencia individual equivalente
Registro	Automático: el prototipo mide, compara y emite el veredicto

2.2 Etapas del proyecto

Etapa	Qué se hace	Resultado documentado	Estado
1. Definición	Problema, hipótesis, variables y umbrales.	Capítulos 1 a 3	**Completado**
2. Investigación	Cómo funcionan los proyectos de referencia.	Capítulo 4	**Completado**
3. Diseño	Alternativas, requerimientos y arquitectura.	Capítulos 5 a 8	**Completado**
4. Desarrollo	Prototipo integrador con pruebas automáticas.	Prototipo v0.2	**Completado**
5. Integración	Conectar la PC y la laptop reales.	Prueba P1	**Pendiente**
6. Experimentación	Pruebas individuales y conjuntas.	Pruebas P2 a P5	**Pendiente**
7. Análisis y decisión	Comparar contra umbrales y decidir.	Capítulo 15	**Pendiente**

2.3 Diagrama 1 – Proceso general del proyecto

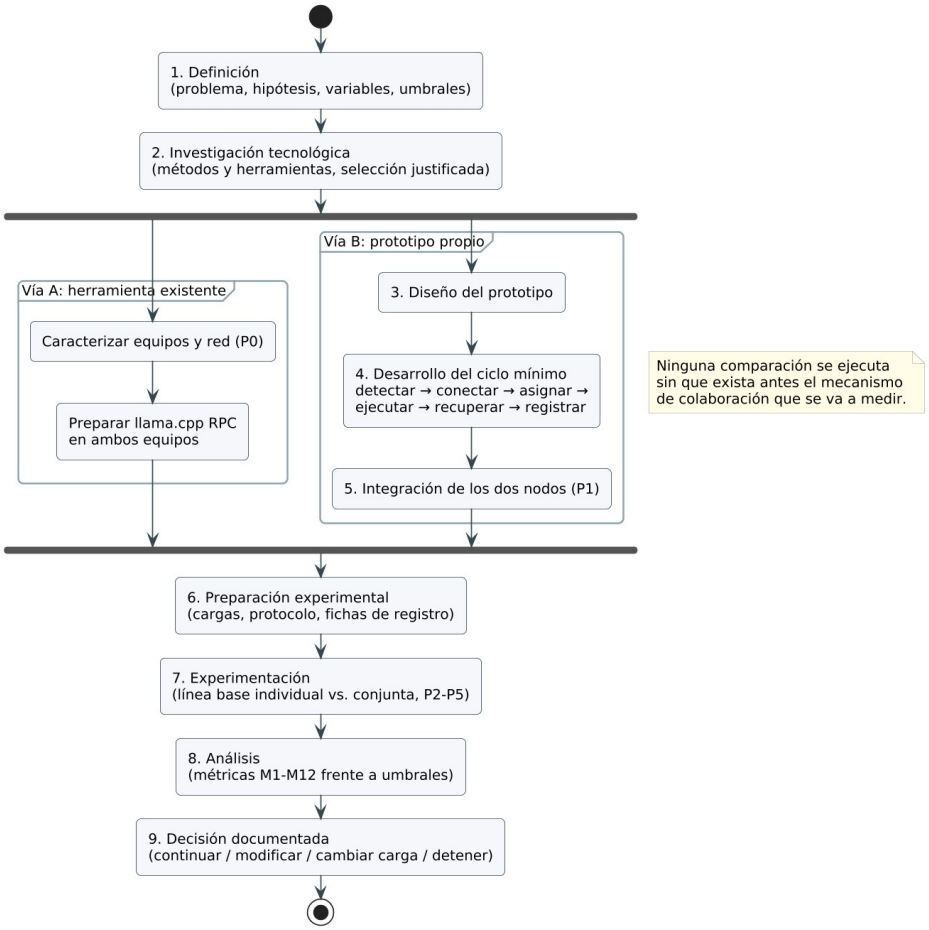


Diagrama 1. Proceso general del proyecto

3. HIPÓTESIS, VARIABLES Y MÉTRICAS

3.1 Hipótesis

Cada hipótesis se acepta o se rechaza con una métrica y un umbral fijados antes de medir. Los umbrales están programados en el prototipo, de modo que el veredicto no depende de la interpretación de quien mide.

ID	Hipótesis	Métrica	Umbral favorable	Umbral desfavorable
H-01	**Colaboración.** Los equipos ejecutan tareas coordinadas y se recuperan de la caída de uno.	M-01, M-02	≥ 95 % de éxito con ambos equipos participando	< 80 %
H-02	**Rendimiento.** En un lote de tareas independientes, dos equipos terminan antes que el mejor equipo solo.	M-06, M-07	Aceleración ≥ 1,3 y eficiencia ≥ 0,8	Aceleración < 1,1
H-03	**Capacidad.** Repartido, se ejecuta un modelo de IA que la PC sola no maneja de forma usable.	M-03, M-08	PC sola < 0,5 tok/s y repartido ≥ 1,5 tok/s	Repartido < 0,5 tok/s o peor que la PC sola
H-04	**Aprovechamiento.** Cada equipo recibe trabajo en proporción a su capacidad real.	M-07	Desvío de cuota ≤ 10 puntos	Desvío > 30 puntos
H-05	**Facilidad.** Una persona sin formación técnica monta XARA con la guía.	M-11	Se registra como línea base	—

Si la variación entre repeticiones supera el 15 %, el resultado se declara **no concluyente**, cualquiera que sea la aceleración.

3.2 Variables independientes

Variable	Valores posibles
Modo de ejecución	Cada equipo solo · todos juntos (el prototipo los ejecuta en secuencia)
Técnica	Cola de tareas (P1, P2, P4) · modelo repartido por capas con llama.cpp RPC (P3)
Carga	Compresión determinista (cpu) · preguntas a un modelo de IA (llm) · espera (sleep)
Modelo de IA	Qwen2.5-Instruct Q4_K_M: 1.5B (≈1 GB) para P4 · 14B (≈9 GB) para P3
Red de la laptop	Wi-Fi · cable (solo si la laptop tiene puerto)

3.3 Variables de control

Deben ser iguales en la ejecución individual y en la conjunta. Las marcadas como automáticas las registra el prototipo; el resto las cuida el usuario.

Variable	Cómo se controla	Registro
Alimentación	Laptop enchufada; el panel avisa si está en batería.	Automático
Carga de fondo	Programas cerrados; el uso de CPU se registra cada 2 s.	Automático
Calentamiento	La primera repetición se ejecuta y se descarta.	Automático
Entrada del modelo	Mismas preguntas, temperatura 0 y semilla fija.	Automático
Software	Misma versión de XARA y de llama.cpp en ambos equipos.	Automático (inventario)

Variable	Cómo se controla	Registro
Wi-Fi	Misma banda y ubicación; la latencia se mide en cada latido.	Automático (latencia)

3.4 Métricas

ID	Métrica	Unidad	Fuente automática
M-01	Tasa de éxito de tareas	%	Tabla tareas
M-02	Reintentos por tarea vencida	n.º	Tabla tareas
M-03	Velocidad de generación de texto	tokens/s	Tabla ia (llama-bench)
M-04	Velocidad de lectura de la entrada	tokens/s	Tabla ia (llama-bench)
M-05	Duración del lote (mediana de repeticiones)	s	Tabla lotes
M-06	Aceleración = mejor equipo solo / todos juntos	—	Resumen de la prueba
M-07	Eficiencia = aceleración / aceleración ideal; cuotas esperada y real	— / %	Resumen de la prueba
M-08	Memoria libre y usada por equipo	GB	Tabla metricas
M-09	Latencia del latido y tráfico de red	ms / MB	Tabla metricas
M-10	Tiempo de arranque del motor y copia del modelo	s	Resumen de la prueba
M-11	Tiempo de instalación y pasos manuales	min / pasos	Bitácora del usuario
M-12	Variación entre repeticiones	%	Resumen de la prueba

4. ESTADO DEL ARTE: CÓMO FUNCIONAN LOS PROYECTOS DE REFERENCIA

Este capítulo resume cómo resuelve cada proyecto su parte del problema, a partir de su documentación, sus artículos y, en el caso de exo, de su código fuente. El objetivo es identificar qué técnica concreta puede reutilizar XARA.

4.1 exo

exo une equipos Apple y Linux para ejecutar un modelo de IA que no cabe en uno solo [3]. Su código muestra tres piezas útiles para XARA:

- **Descubrimiento.** Cada equipo anuncia su presencia por multidifusión UDP en la red local con una marca propia («EXO») y se conecta a los que encuentra; no se escriben direcciones. Entre los equipos se elige un coordinador por votación.
- **Ubicación del modelo.** Busca el grupo más pequeño de equipos cuya memoria libre total alcanza para el modelo, prefiere los que ya tienen el modelo descargado y, a igualdad, los de más memoria.
- **Reparto de capas.** La función allocate_layers_proportionally asigna las capas en proporción a la memoria de cada equipo, con un mínimo de una capa por equipo.

Limitaciones: su motor es MLX y funciona con GPU en macOS y solo con CPU en Linux; Windows no figura entre los sistemas admitidos [3]. El artículo de prima.cpp señala además que repartir solo por memoria no siempre es óptimo, porque más memoria no implica más velocidad [2].

4.2 llama.cpp en modo RPC

Cada equipo ayudante ejecuta un rpc-server que expone su CPU o su GPU. El equipo principal arranca el modelo con la opción --rpc y la lista de ayudantes; por defecto, los pesos y la memoria de contexto se reparten en proporción a la memoria disponible, y la proporción puede fijarse con --tensor-split. La opción -c guarda en el ayudante una copia local de los pesos para no reenviarlos en cada carga [1].

Es el único motor maduro para partir un modelo entre equipos con Windows, por lo que XARA lo usa en lugar de reimplementarlo. Su documentación lo califica como prueba de concepto frágil e insegura, y advierte que no debe usarse en redes abiertas [1].

4.3 Incredibuild

Acelera compilaciones mediante **virtualización de procesos**. El equipo que inicia el trabajo intercepta la creación de procesos (por ejemplo, cada llamada al compilador) y los envía a núcleos libres de otros equipos de la red, los ayudantes. En el ayudante, el proceso remoto «ve» el sistema de archivos y el registro del equipo que inició el trabajo: los archivos que necesita se piden bajo demanda, comprimidos y en caché, y el resultado vuelve como si se hubiera ejecutado en local. Según el fabricante, no hace falta instalar la aplicación en los ayudantes [8].

XARA toma la idea de **enviar los archivos al ayudante solo cuando los necesita**: el modelo de IA se copia a la laptop la primera vez que hace falta. La virtualización completa de procesos queda fuera del alcance de esta versión.

4.4 Colas de tareas: BOINC, HTCondor, Flamenco y Dask

Proyecto	Cómo reparte	Qué hace si un equipo falla	Qué toma XARA
BOINC [9]	Cada cliente pide trabajo al servidor por HTTP cuando lo necesita (pull); respeta las preferencias del dueño del equipo.	Cada unidad tiene un plazo; si no vuelve a tiempo, se reasigna. Valida resultados por redundancia.	Reparto pull y plazo con reasignación.
HTCondor [10]	Empareja los requisitos de cada trabajo con los anuncios de las máquinas (ClassAds); aprovecha escritorios ociosos.	Reencola el trabajo en otra máquina.	Inventario anunciado por cada equipo.
Flamenco [11]	Un Manager con base SQLite reparte tareas de render a los Workers; requiere almacenamiento compartido y escribir la dirección del Manager.	La tarea vuelve a la cola.	Coordinador con SQLite y panel web.
Dask [12]	El planificador asigna tareas y los equipos libres «roban» trabajo a los ocupados.	Reprograma las tareas del equipo caído.	Referencia; funciona en Windows pero exige programar.

4.5 Otros proyectos revisados

Proyecto	Cómo funciona	Limitación para el usuario objetivo
Petals [4]	Cada servidor guarda un tramo de capas y lo anuncia en una tabla distribuida; el cliente elige la cadena de servidores de menor latencia y, si uno cae, rehace la ruta y reenvía la entrada.	Windows solo mediante WSL; pensado para GPU.
prima.cpp [2]	Anillo de equipos que precarga tramos de capas desde disco; un planificador asigna capas según RAM, VRAM y velocidad de disco. Tolera Wi-Fi.	No documenta soporte de Windows.
GPUStack [7]	Servidor central y trabajadores que aportan GPU a un fondo común.	Los trabajadores solo funcionan en Linux; instalación con Docker.
Juice [13]	GPU por red: un servidor comparte su GPU y las aplicaciones de otros equipos la usan sin modificarse.	Comparte una GPU, no suma equipos; versión comunitaria.
Ray [5]	Clúster de cómputo distribuido para Python.	No admite clústeres de varios equipos en Windows.
distributed-llama [6]	Paralelismo tensorial: cada capa se divide entre equipos.	Requiere 2, 4, 8… equipos y red por cable.
LM Link [14]	Usa desde un equipo los modelos de otro por un enlace cifrado.	No reparte un modelo; solo acceso remoto.

4.6 Qué toma XARA de cada proyecto

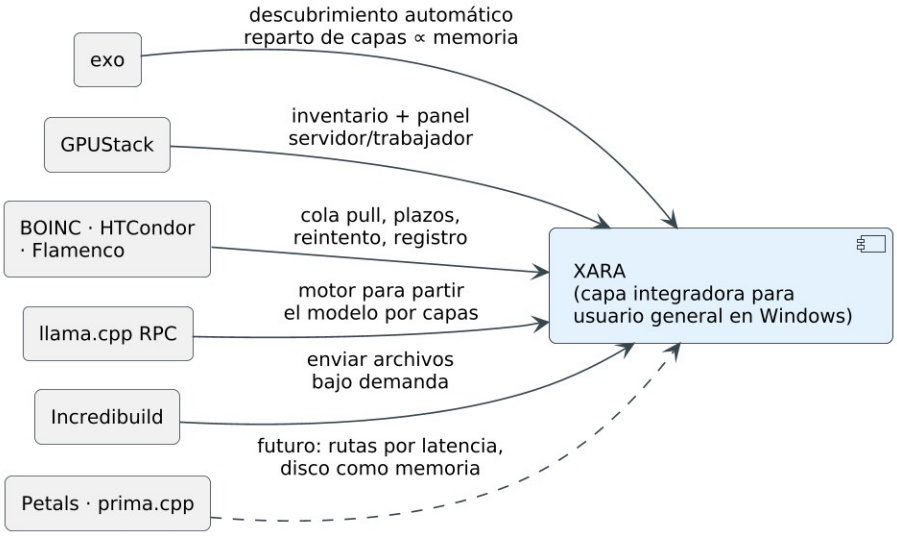


Diagrama 8. Técnicas probadas que integra XARA

Conclusión. Cada técnica necesaria ya existe y está demostrada. Ninguno de estos proyectos las reúne en un programa que un usuario general instale en Windows con doble clic y que, además, le diga con datos si le conviene repartir. Ese es el hueco que ocupa XARA.

5. ALTERNATIVAS Y DECISIÓN DE CONSTRUCCIÓN

5.1 Métodos de distribución

Las formas de colaborar no compiten entre sí: resuelven problemas distintos. Elegir el método equivocado es la causa principal de que dos equipos rindan menos que uno.

Método	Qué reparte	Sensibilidad a la red	Ejemplos	Uso en XARA
Cola de tareas	Tareas independientes	Baja	BOINC, HTCondor, Flamenco, Dask	Implementado (P1, P2, P4)
Por capas (pipeline)	Un modelo: cada equipo guarda un tramo	Baja-media	llama.cpp RPC, exo, Petals, prima.cpp	Implementado con llama.cpp (P3)
Tensorial	Un modelo: cada capa se divide	Alta	distributed-llama, exo	Descartado con Wi-Fi
Virtualización de procesos	Procesos de un programa	Media	Incredibuild	Futuro
GPU por red	Acceso a una GPU remota	Alta	Juice	Futuro

5.2 Modelo de rendimiento

El tiempo para generar cada palabra con un modelo repartido puede estimarse como la suma del tiempo de lectura de memoria de cada equipo más el de los viajes de red. Lo que viaja en cada paso es muy pequeño, por lo que pesa la latencia y no el caudal.

$$t \approx \Sigma\ (W / B) + v \times L$$

W es el tamaño del tramo del modelo en cada equipo, B el ancho de banda de su memoria, v el número de viajes de red por palabra y L la latencia de un viaje.

Caso (modelo de ≈14B, ≈9 GB)	Viajes por palabra	Coste de red con Wi-Fi	Peso sobre el total	Conclusión
Por capas, 2 equipos	≈ 2	6–20 ms	Menos del 5 %	El Wi-Fi no es el cuello de botella
Tensorial, 2 equipos	≈ 96	0,3–1 s	Duplica o triplica el tiempo	Inviable con Wi-Fi

Repartir por capas no acelera, porque los tramos se ejecutan uno detrás de otro; lo que gana es **capacidad**. La aceleración se busca con tareas independientes.

5.3 Alternativas para construir XARA

Alternativa	Ventaja	Inconveniente	Decisión
A. Construir todo desde cero	Control total	Repite trabajo ya resuelto; meses antes de medir nada	Descartada
B. Adaptar exo	Descubrimiento y reparto ya hechos	Motor MLX sin Windows; base grande en Python y Rust	Descartada
C. Montar sobre Ray o Dask	Planificadores maduros	Ray no admite varios equipos en Windows; Dask exige programar	Descartada

Alternativa	Ventaja	Inconveniente	Decisión
D. Capa integradora propia sobre motores existentes	Programa pequeño, funciona en Windows, usa llama.cpp para la IA	Depende de la evolución de llama.cpp	**Elegida**
E. Usar herramientas sueltas a mano	Nada que programar	No es un producto; registro manual	Solo como referencia

5.4 Registro de decisiones tecnológicas

Componente	Decisión	Motivo	Riesgo aceptado
Motor de IA repartida	llama.cpp RPC	Única opción madura con Windows; reparte por capas.	Frágil e inseguro según su documentación.
Reparto de tareas	Pull con plazo	Equilibra solo equipos desiguales (BOINC, HTCondor).	Más mensajes de control.
Ubicación del modelo	Según RAM libre, reparto proporcional	Criterio de exo; simple y verificable.	No considera la velocidad de cada equipo.
Descubrimiento	Difusión UDP + dirección manual	Funciona sin instalar nada.	Algunos routers la bloquean.
Comunicación	HTTP-JSON	Se inspecciona con un navegador.	Sin cifrado.
Registro	SQLite + informes + CSV	Automático, consultable y exportable a Excel.	—
Lenguaje	Python	Instalación sencilla en Windows; el trabajo pesado lo hace llama.cpp.	Coordinador menos eficiente.

6. REQUERIMIENTOS DEL PROTOTIPO

6.1 Requerimientos funcionales

ID	Requerimiento	Descripción	Prior.	Estado
RF-01	Descubrir equipos	El primer equipo que arranca es coordinador; los demás lo encuentran por UDP. Dirección manual como respaldo.	Alta	**Implementado**
RF-02	Inventario	Cada equipo informa CPU, AVX2, RAM, GPU, discos, sistema y motores de llama.cpp.	Alta	**Implementado**
RF-03	Latido y métricas	Cada 2 s: CPU, RAM, red, batería y latencia. 10 s sin latido = equipo inactivo.	Alta	**Implementado**
RF-04	Cola pull	Cada equipo pide tarea cuando queda libre; se puede restringir un lote a ciertos equipos.	Alta	**Implementado**
RF-05	Recuperar fallos	Una tarea que no vuelve en el plazo (300 s) regresa a la cola y se cuenta el reintento.	Alta	**Implementado**
RF-06	Planificar el modelo	Decide «un solo equipo», «repartir por capas» o «no cabe» según la RAM libre, con margen de 1 GB.	Alta	**Implementado**
RF-07	Repartir el modelo	Arranca rpc-server en cada equipo y ejecuta el modelo con reparto proporcional.	Alta	**Implementado**
RF-08	Copia bajo demanda	Copia el modelo del coordinador al equipo que no lo tiene.	Media	**Implementado**
RF-09	Pruebas automáticas	P1 a P4 con referencia individual, calentamiento y repeticiones.	Alta	**Implementado**
RF-10	Registro automático	Base SQLite, informe por prueba y exportación CSV.	Alta	**Implementado**
RF-11	Panel web	Equipos, recursos en vivo, botones de prueba, veredictos y descargas.	Media	**Implementado**
RF-12	Instalación en Windows	Lanzador, script de cortafuegos e instalador de llama.cpp y modelos con doble clic.	Alta	**Parcial**
RF-13	Funcionar en los equipos reales	Ciclo completo entre la PC y la laptop.	Alta	**Pendiente**

6.2 Requerimientos no funcionales

ID	Categoría	Criterio / medida	Estado
RNF-01	Instalación	Solo requiere Python; psutil se instala solo y es opcional.	**Implementado**
RNF-02	Tolerancia a fallos	Ninguna tarea se pierde si un equipo se desconecta.	**Implementado**
RNF-03	Trazabilidad	Cada tarea, lote y métrica queda registrada con su equipo y su prueba.	**Implementado**
RNF-04	Facilidad	Guía no técnica; sin escribir direcciones ni comandos en el caso normal.	**Implementado**
RNF-05	Sobrecoste	La coordinación consume menos del 5 % del tiempo del lote.	**Pendiente**
RNF-06	Seguridad	Uso exclusivo en red privada; sin autenticación en v0.2.	**Parcial**
RNF-07	Portabilidad	Mismo programa en Windows y Linux.	**Parcial**

7. ANÁLISIS FUNCIONAL Y DIAGRAMAS

7.1 Diagrama 2 – Casos de uso

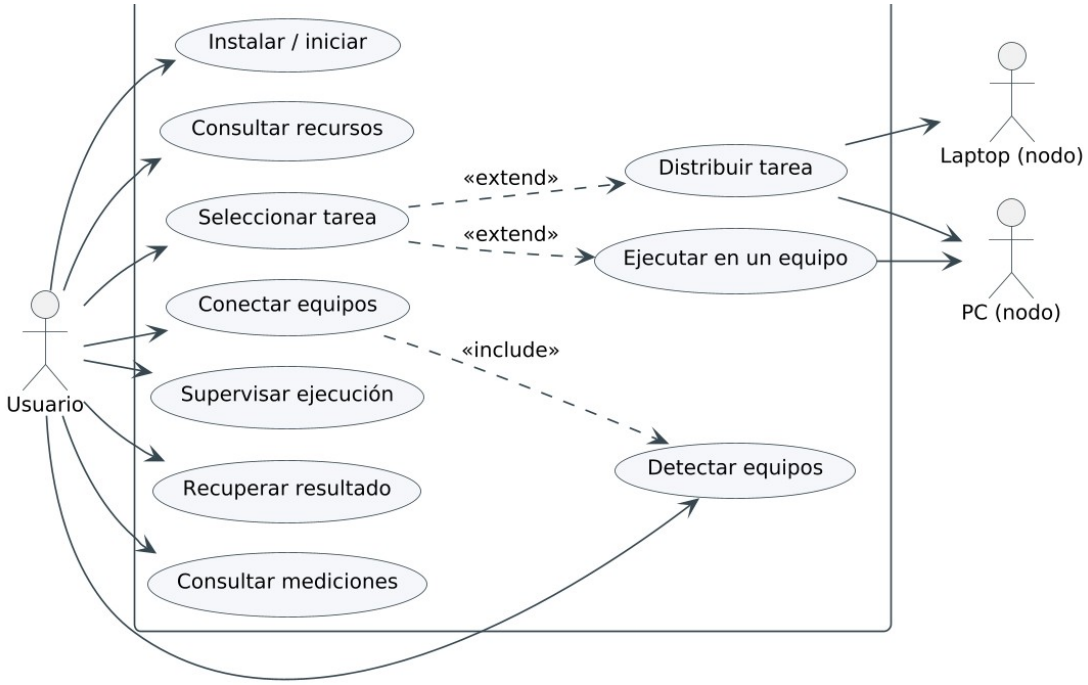


Diagrama 2. Casos de uso del usuario

7.2 Diagrama 4 – Secuencia de una prueba

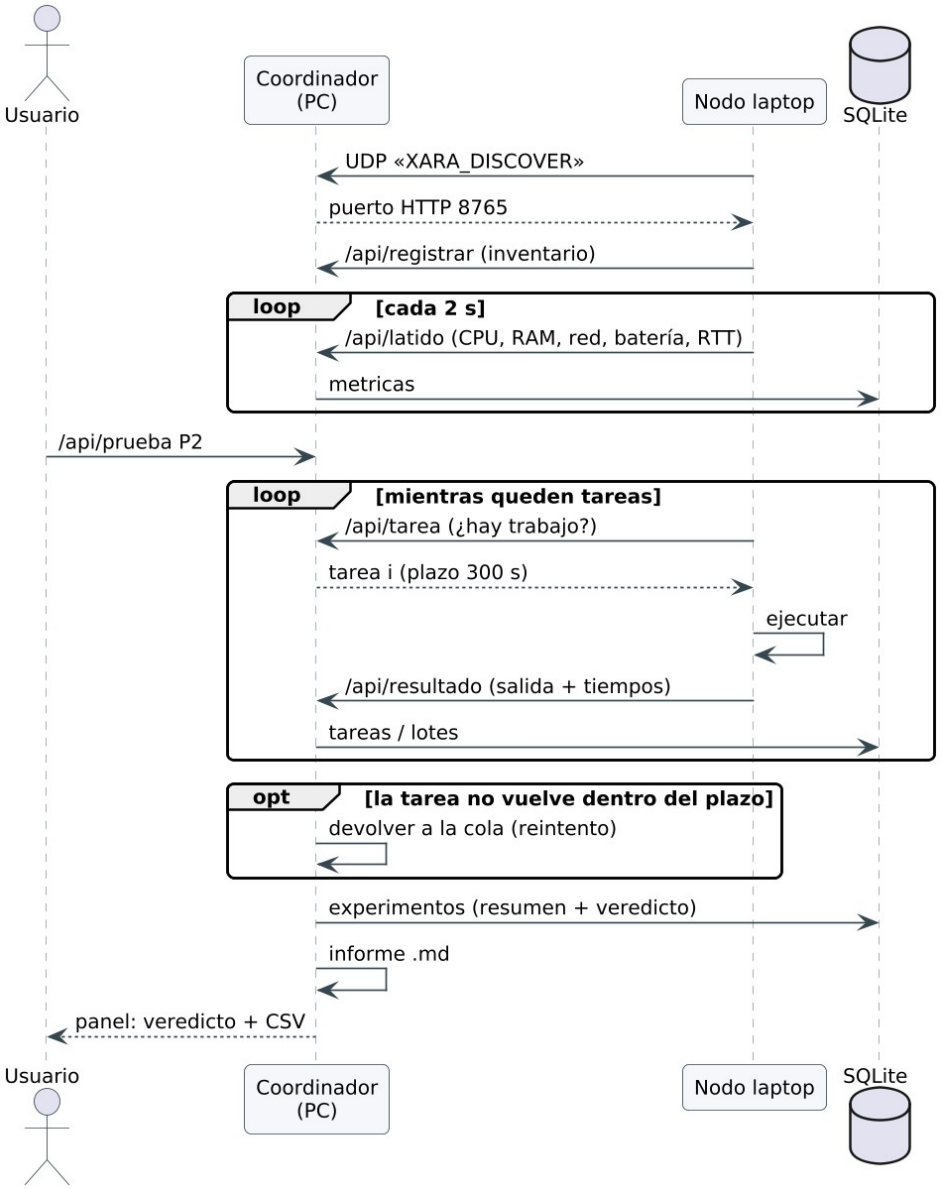


Diagrama 4. Descubrimiento, latido, reparto pull y registro automático

8. ARQUITECTURA DEL PROTOTIPO V0.2

8.1 Componentes

Componente	Función
Coordinador	Mantiene la lista de equipos, la cola de tareas y los plazos; planifica el modelo; ejecuta las pruebas.
Nodo	Se ejecuta en cada equipo, incluido el coordinador. Envía latidos, pide tareas y arranca o para llama.cpp.
Motores	llama-bench, llama-server y rpc-server de llama.cpp, en la carpeta motores de cada equipo.
Registro	Base SQLite datos/xara.db, informes en datos/informes y exportación CSV.
Panel	Página web servida por el coordinador en el puerto 8765.

8.2 Diagrama 3 – Arquitectura

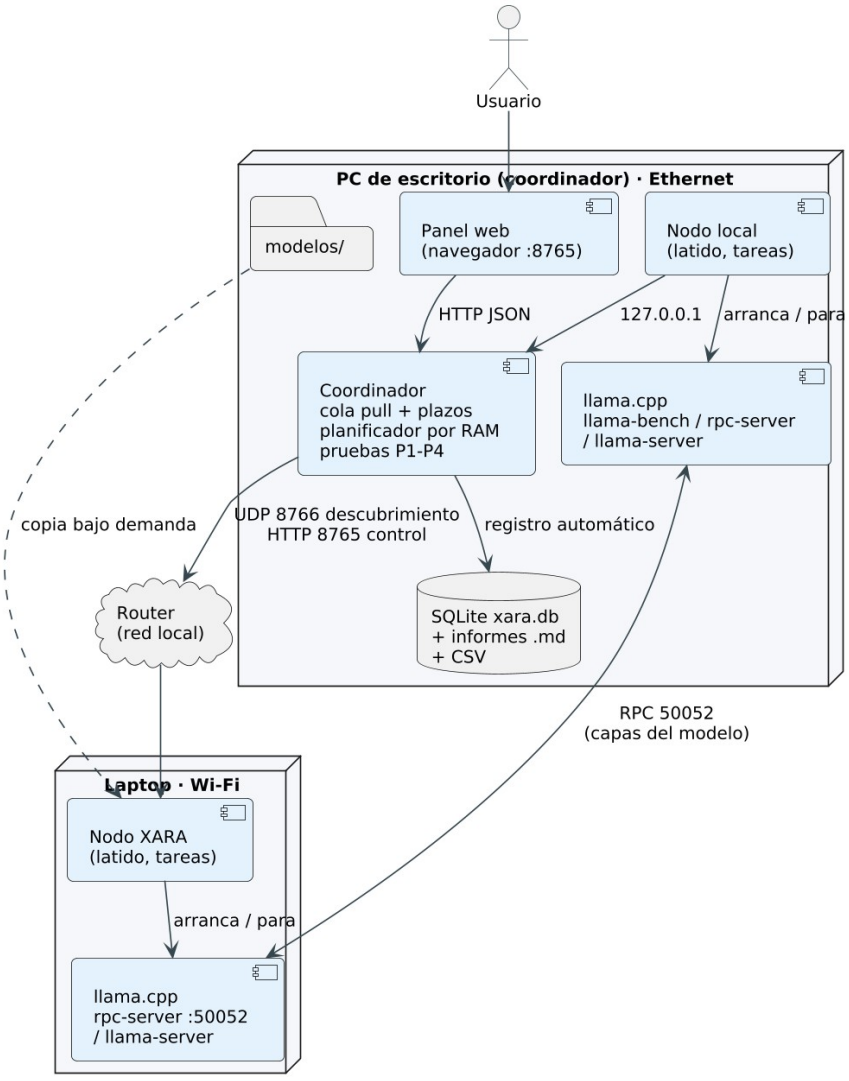


Diagrama 3. Arquitectura del prototipo v0.2

8.3 Puertos y mensajes

Puerto / mensaje	Dirección	Contenido
UDP 8766	Equipo → red	Búsqueda del coordinador; responde con su puerto HTTP
POST /api/registrar	Nodo → coordinador	Inventario del equipo
POST /api/latido	Nodo → coordinador	Métricas cada 2 s y latencia del latido anterior
GET /api/tarea	Nodo → coordinador	Petición de trabajo; respuesta vacía si no hay
POST /api/resultado	Nodo → coordinador	Resultado, error y tiempo de ejecución
POST /api/prueba	Panel → coordinador	Inicio de P1, P2, P3 o P4 con sus opciones
GET /api/estado, /api/experimentos, /api/informe, /api/csv	Panel → coordinador	Estado en vivo, resultados, informes y datos
GET /modelos/<archivo>	Nodo → coordinador	Copia del modelo bajo demanda
TCP 50052	llama.cpp ↔ llama.cpp	Capas del modelo repartido (rpc-server)

8.4 Datos registrados

Tabla	Una fila por	Campos principales
nodos	Equipo	Nombre, IP, inventario completo, primera y última conexión
metricas	Latido (2 s)	CPU, RAM usada y libre, red enviada y recibida, latencia, batería, prueba
tareas	Tarea	Lote, equipo, tipo, tiempos de envío y fin, ejecución, reintentos, error, salida
lotes	Lote	Etiqueta, total, correctas, fallidas, duración y reparto por equipo
experimentos	Prueba	Tipo, inicio, fin, estado, opciones, resumen y veredicto
ia	Medición de IA	Configuración, modelo, reparto, tokens/s de entrada y salida, error

9. PLAN EXPERIMENTAL

9.1 Protocolo común

- Cada prueba se lanza con un botón del panel; el prototipo ejecuta las configuraciones, mide y emite el veredicto.
- Las pruebas de rendimiento ejecutan el mismo lote en cada equipo solo y en todos juntos, con una repetición de calentamiento que se descarta.
- Se informa la mediana de las repeticiones y su variación.
- Antes de cada sesión, el usuario cierra programas y enchufa la laptop; el panel avisa si está en batería.

9.2 Matriz de pruebas

ID	Objetivo	Qué hace el prototipo	Valores por defecto	Hipótesis
P0	Conocer los equipos	Inventario al conectarse (CPU, AVX2, RAM, GPU, discos)	—	—
P1	Validar el ciclo y conocer cada CPU	Envía tareas de espera; mide éxito, latencia, AVX2 y velocidad de un núcleo por equipo	20 tareas de 0,5 s	H-01
P2	Aceleración en tareas independientes	Lote de compresión: cada equipo solo y ambos	24 tareas de 8 MB, 3 repeticiones + 1	H-02, H-04
P3	Capacidad con modelo repartido	Planifica por RAM; mide el coordinador solo y el modelo repartido	Entrada 64, salida 32, 3 repeticiones	H-03
P4	Aceleración en IA sin repartir el modelo	Arranca la IA en cada equipo y reparte 10 preguntas	48 palabras por respuesta	H-02
P5	Reproducibilidad	Repetir P2 a P4 otro día	Los mismos	Todas

La recuperación ante fallos (H-01) se comprueba cerrando XARA en la laptop durante P1: la tarea vencida vuelve a la cola y el reintento queda registrado. Para no esperar 300 s, el plazo se reduce con la variable XARA_LEASE_S. La velocidad de P1 mide un núcleo con compresión zlib, que no usa AVX2; sirve para comparar equipos, pero el efecto de AVX2 en la IA solo se ve en P3 y P4. El ancho de banda de memoria (winsat mem) aún no se mide de forma automática.

9.3 Resultados esperados

Son estimaciones razonadas, no datos. Se fijan antes de medir para no interpretar los resultados a conveniencia.

Prueba	Resultado esperado	Fundamento
P2	Aceleración entre 1,5 y 2,0	Cuatro núcleos en ambos equipos; la laptop tiene 8 hilos pero consumo limitado a 15 W.
P3 – PC sola, 14B	No cabe o usa el disco: < 0,5 tok/s	≈9 GB de modelo frente a ≈8 GB libres.
P3 – ambos, 14B	Entre 1,2 y 2,2 tok/s	Modelo de rendimiento del apartado 5.2.
P3 – ambos, modelo pequeño	Igual o algo más lento que la PC sola	Repartir por capas no acelera.
P4	Aceleración entre 1,5 y 1,9	Cada equipo genera en paralelo con su propio modelo.

9.4 Diagrama 5 – Primer experimento

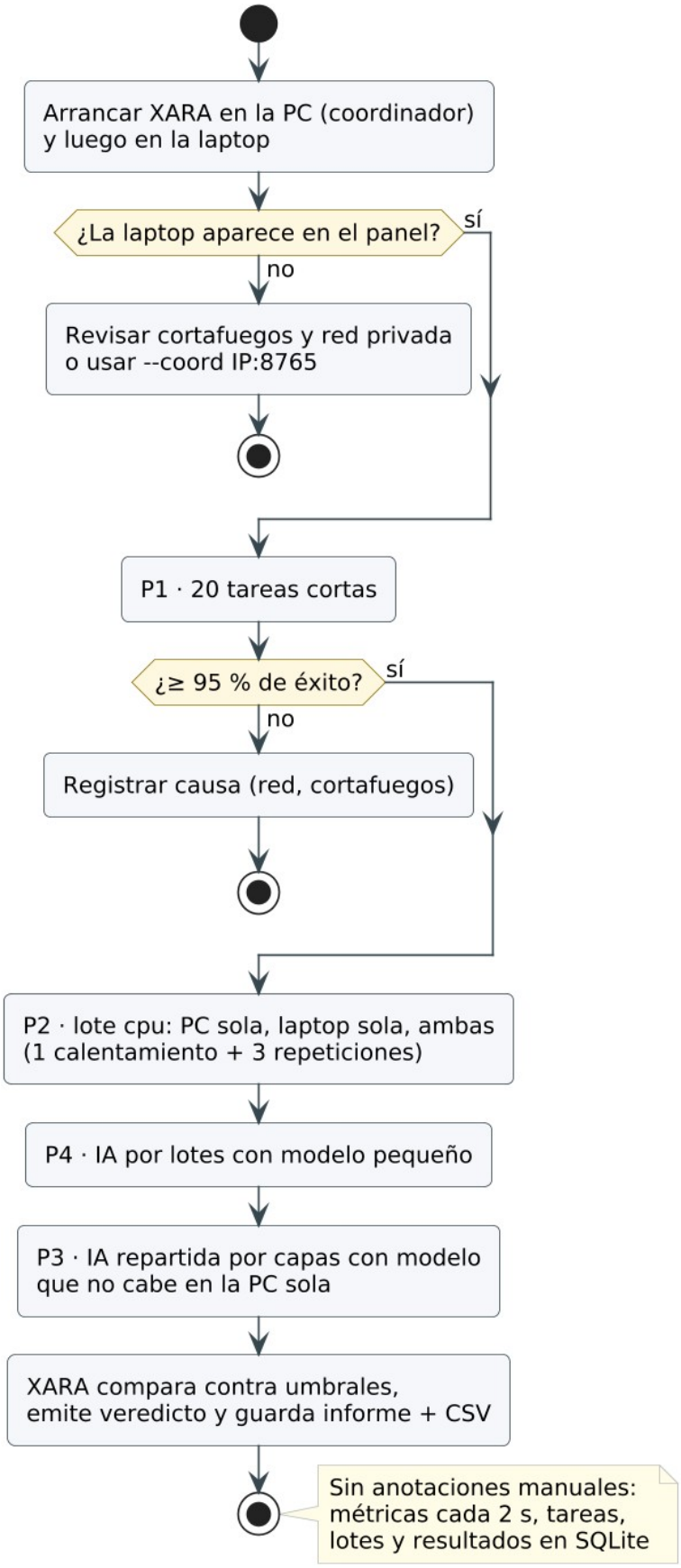


Diagrama 5. Secuencia del primer experimento con los equipos reales

10. EVALUACIÓN Y DECISIÓN

10.1 Escala de resultados

Resultado	Condición
Favorable	La hipótesis alcanza su umbral favorable y P5 lo confirma con una variación de hasta 15 %.
Favorable con condiciones	El umbral solo se alcanza con una configuración concreta, por ejemplo con cable.
Técnicamente viable	La colaboración funciona, pero el resultado queda entre los dos umbrales.
Desfavorable	Se alcanza el umbral desfavorable.
No concluyente	La variación supera el 15 % o no puede aislarse la causa.
Inválido	La prueba falló o cambió una variable de control.

10.2 Regla de interpretación

Un resultado positivo debe poder atribuirse a la colaboración y no a otra variable. Un resultado negativo también es evidencia válida y se documenta con su causa probable: la carga, el hardware, la red o la herramienta.

10.3 Diagrama 6 – Árbol de decisión

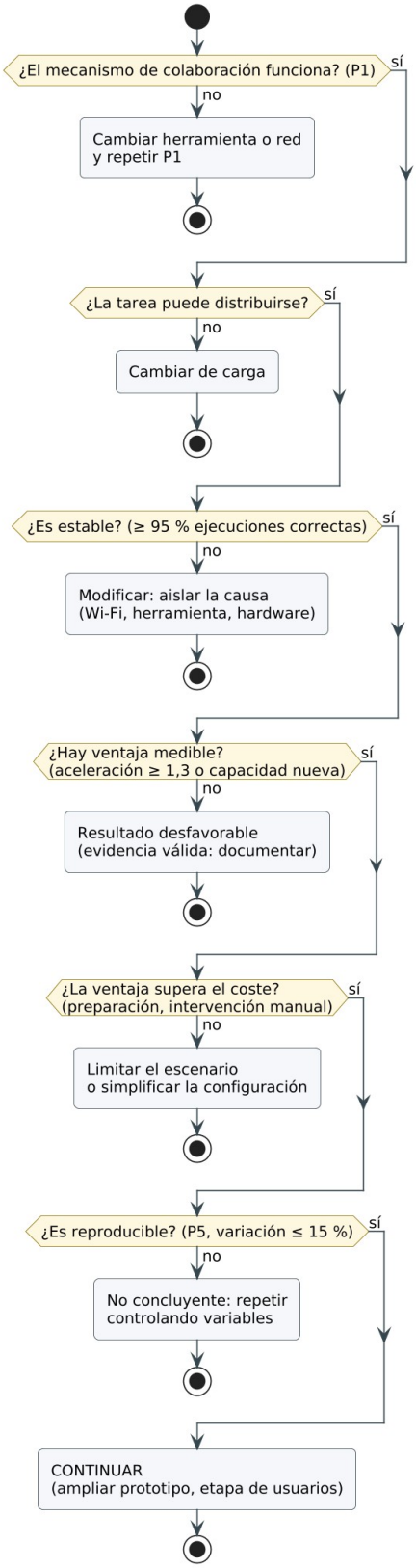


Diagrama 6. Árbol de decisión

11. PLANIFICACIÓN Y CRONOGRAMA

11.1 Hitos principales

Hito	Resultado previsto	Estado
H1	Problema, hipótesis, variables y umbrales definidos	**Completado**
H2	Proyectos de referencia estudiados y alternativas comparadas	**Completado**
H3	Diseño del prototipo	**Completado**
H4	Prototipo v0.2 con pruebas y registro automáticos, verificado en laboratorio	**Completado**
H5	Instalación en la PC y la laptop (guía)	**Pendiente**
H6	Dos equipos comunicados en Windows (P1)	**Pendiente**
H7	Comparaciones individual y conjunta (P2 a P4)	**Pendiente**
H8	Repetición (P5) y análisis	**Pendiente**
H9	Decisión de continuidad	**Pendiente**

11.2 Diagrama 7 – Cronograma de Gantt

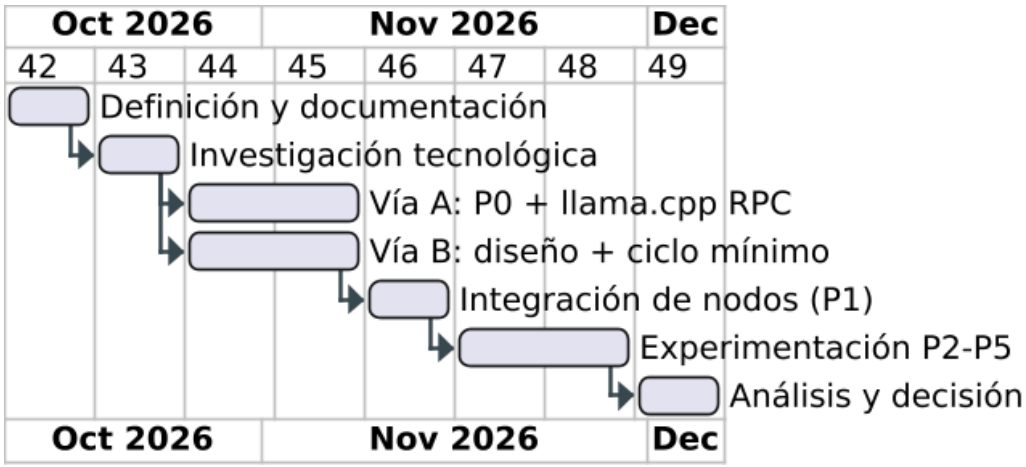


Diagrama 7. Cronograma provisional, ajustable según los resultados de cada etapa

12. RIESGOS

Riesgo	Señal de alerta	Respuesta
Windows bloquea la comunicación	La laptop no aparece en el panel	Script de cortafuegos, red privada y dirección manual.
Memoria de la laptop en un solo canal	Velocidad de IA baja en la laptop	Asignarle menos capas y registrarlo como límite del hardware.
Wi-Fi inestable	Latencia mayor de 50 ms o cortes en el panel	Usar 5 GHz, acercar la laptop al router o conectar por cable.
El modelo grande no cabe ni repartido	Plan «no cabe» o error de memoria en P3	Usar un modelo intermedio o estudiar prima.cpp.
Cambios en llama.cpp	Fallan rpc-server o llama-bench tras actualizar	Usar la misma versión en ambos equipos y registrarla.
Función de otro producto	LM Studio u Ollama añaden reparto entre equipos	Revisión trimestral; diferenciar en facilidad y medición.
Éxito aislado	Variación alta en P5	Repetir; no concluir con una sola ejecución.

13. REGISTRO AUTOMÁTICO Y EVIDENCIAS

13.1 Regla de trazabilidad

Toda conclusión debe enlazarse con una prueba registrada. El prototipo genera la evidencia sin intervención: cada prueba tiene un identificador, sus opciones, las métricas de cada equipo durante la prueba, el resumen y el veredicto. Lo que no tenga registro se trata como hipótesis.

13.2 Contenido del informe de cada prueba

Sección	Contenido
Cabecera	Identificador, tipo de prueba, fecha, estado y veredicto
Equipos	Inventario de cada equipo participante
Recursos	CPU, RAM, latencia, red y batería durante la prueba, por equipo
Resultado	Resumen completo: duraciones, aceleración, eficiencia, cuotas, tokens/s, plan del modelo
Pasos	Lo que hizo el prototipo, en orden, con su hora

13.3 Verificación del prototipo

El prototipo se verificó en Linux, con el coordinador y un segundo equipo simulados en la misma máquina, llama.cpp compilado desde su código fuente y un modelo de prueba de 115 MB con pesos aleatorios. Esta verificación comprueba que el ciclo funciona; sus cifras no son representativas de los equipos reales.

Prueba	Comprobación	Resultado
P1	Ambos equipos ejecutaron tareas; 100 % de éxito	**Completado**
P2	Referencias individuales y conjunta con veredicto automático	**Completado**
P3	Reparto real del modelo con rpc-server en ambos equipos y plan automático	**Completado**
P4	Copia del modelo bajo demanda y respuestas en ambos equipos	**Completado**
Windows	Ejecución en la PC y la laptop reales	**Pendiente**

13.4 Entregables

Código	Entregable	Ubicación	Estado
E1	Estado del arte y alternativas	Capítulos 4 y 5	**Completado**
E2	Diseño del prototipo	Capítulos 6 a 8	**Completado**
E3	Prototipo v0.2	xara/prototipo	**Completado**
E4	Guía no técnica paso a paso	xara/prototipo/GUIA.md	**Completado**
E5	Diagramas editables	xara/diagramas	**Completado**
E6	Registros de las pruebas reales	datos/ en la PC	**Pendiente**
E7	Comparación individual y conjunta	Informes de P2 a P5	**Pendiente**
E8	Decisión final	Capítulo 15	**Pendiente**

14. MERCADO, CASOS DE USO Y ESTUDIO ECONÓMICO

Todo este capítulo es una hipótesis. No hay todavía mediciones en los equipos reales ni conversaciones con clientes. Se valida en dos pasos: primero, que P3 y P4 muestren una mejora real; después, entrevistas con cinco clientes posibles.

14.1 Principio: qué se puede repartir

XARA solo ayuda cuando el trabajo puede partirse en piezas que no necesitan hablar entre sí a cada instante: muchos archivos, muchos fotogramas, muchas preguntas o un modelo de IA dividido por capas. Un programa que funciona como un único proceso, como Word, Chrome o un videojuego, no se acelera repartiéndolo sin reescribirlo. Es un límite técnico, no del diseño de XARA.

14.2 Usos por tipo de usuario

Usuario	Ejemplo de tarea	Por qué se puede repartir	Herramienta que ya lo hace	Estado en XARA
Hogar con PCs viejas	Usar un asistente de IA de 14B que no cabe en una sola PC; convertir de una vez la colección de videos familiares.	El modelo se parte por capas; cada video es independiente.	exo, llama.cpp RPC	IA: P3. Videos: futuro
Editor de video	Exportar 40 clips o generar sus versiones ligeras mientras sigue editando.	Cada clip se procesa por separado.	Apple Compressor (solo Mac, con equipos compartidos)	Futuro: conector de ffmpeg
Estudio 3D	Renderizar una animación de 1.000 fotogramas en Blender.	Cada fotograma es independiente.	Flamenco, Deadline	Futuro: conector de Blender
Programador	Compilar un proyecto grande de C++.	Cada archivo se compila por separado.	Incredibuild, distcc	Futuro
Colegio o laboratorio	Usar las 20 PCs del aula en horas libres para IA local o simulaciones de los alumnos.	Cada alumno o simulación es una tarea independiente.	HTCondor, BOINC	Cola de tareas: P2
Pyme	Asistente de IA privado sobre sus documentos y lectura automática de facturas escaneadas, sin subir datos a la nube.	Cada documento es independiente; el modelo puede partirse.	Ollama o LM Studio (un solo equipo)	IA: P3 y P4
Entusiasta de IA local	Ejecutar un modelo de 30B con dos o tres PCs, o responder más preguntas por minuto.	Reparto por capas o por lotes de preguntas.	exo, GPUStack, llama.cpp RPC	P3 y P4

La columna «Estado en XARA» indica qué prueba del prototipo cubre cada caso. «Futuro» significa que XARA necesita un conector para lanzar ese programa (Blender, ffmpeg o el compilador); el reparto de la cola ya existe.

14.3 Usos que no sirven

Uso	Razón técnica	Alternativa que sí existe
Videojuegos	A 60 imágenes por segundo, cada una debe estar lista en unos 16 ms. Un viaje por Wi-Fi ya consume varios milisegundos, y los juegos no están hechos para repartirse entre máquinas.	Jugar en la laptop con el juego corriendo en la PC: Steam Remote Play, Moonlight.
Programas comunes (Word, Chrome, Excel)	Funcionan como un solo proceso; repartirlos exigiría reescribirlos.	Ninguna necesaria: ya funcionan bien en un equipo.

Uso	Razón técnica	Alternativa que sí existe
«Sumar RAM»	Leer la memoria propia tarda unos 100 ns; pedir un dato a otro equipo por la red tarda alrededor de 1 ms, unas 10.000 veces más.	Repartir el trabajo, no la memoria (capítulo 5).

14.4 Segmentos y disposición a pagar

Segmento	Qué gana	Disposición a pagar (hipótesis)	Prioridad
Hogar	Usar modelos de IA más grandes sin comprar hardware	Baja: paga poco y tiene alternativas gratis	Usuario de la versión gratis
Profesionales: video, 3D, programación	Tiempo de espera, que para ellos es dinero	Media-alta: Incredibuild demuestra que se paga por esto	Alta
Colegios y laboratorios	Aprovechar equipos ya comprados	Baja por persona; posible licencia por institución	Media
Pymes	IA privada sin nube y procesos por lotes	Media	Media

14.5 Modelo de negocio posible

Versión	Para quién	Qué incluiría
Gratis	Uso personal y hogar	Descubrimiento automático, IA local repartida, cola de tareas y panel.
Pago	Profesionales, pymes e instituciones	Más equipos, conectores para Blender, ffmpeg y compilación, varios usuarios y soporte.

Para una sola persona, lo realista es un negocio pequeño y rentable en un nicho profesional. Crecer más allá depende de datos que aún no existen.

14.6 Alternativas del usuario

Alternativa	Ventaja principal	Limitación
Usar un solo equipo	Simplicidad	Capacidad limitada
Comprar hardware	Aumento directo de capacidad	Inversión inicial
Usar la nube	Hardware potente por horas	Coste recurrente, dependencia de Internet y privacidad
Acceso remoto (LM Link, Ollama)	Usa el mejor equipo de la casa desde otro	No suma capacidad
Herramientas técnicas (exo, GPUStack, HTCondor)	Potentes y gratuitas	Exigen conocimientos; varias sin Windows
XARA	Aprovecha lo que ya existe, en local, con medición	Límites de la red y del hardware doméstico

14.7 Cómo se valida

Paso	Qué se hace	Criterio para seguir
1	Ejecutar P3 y P4 en la PC y la laptop.	H-02 o H-03 favorables en al menos un escenario.
2	Entrevistar a cinco clientes posibles de los segmentos profesionales.	Al menos tres confirman el problema y que hoy pierden tiempo por él.
3	Elegir un nicho y construir su conector (por ejemplo, Blender o ffmpeg).	Mejora medida con su propio trabajo, no con cargas de prueba.

15. CIERRE Y APROBACIÓN

15.1 Pendientes

N.º	Pendiente	Depende de	Estado
1	Instalar Python y ejecutar el script de cortafuegos en ambos equipos (guía, paso 1 a 3).	LM	**Pendiente**
2	Ejecutar P1 y P2 entre la PC y la laptop.	Punto 1	**Pendiente**
3	Ejecutar instalar_ia.bat en ambos equipos (llama.cpp) y descargar en la PC los modelos pequeño y grande.	Internet y espacio en disco	**Pendiente**
4	Ejecutar P4 y P3, y repetir otro día (P5).	Punto 3	**Pendiente**
5	Confirmar si la laptop tiene puerto de red por cable.	LM	**Pendiente**
6	Anotar el tiempo de instalación y los pasos con dudas (H-05).	LM	**Pendiente**
7	Entrevistar a cinco clientes posibles (apartado 14.7).	Punto 4	**Pendiente**

15.2 Acta de resultado

Elemento	Registro final
Fecha de cierre	
Pruebas ejecutadas	
Resultado por hipótesis	
Principal resultado positivo	
Principal limitación	
Evidencia principal	
Decisión	Continuar · modificar · cambiar de carga · detener
Justificación	
Siguiente etapa	

ANEXOS

Anexo A – Fuentes

[1] llama.cpp, documentación del modo RPC. github.com/ggml-org/llama.cpp/tree/master/tools/rpc

[2] Li et al., «Prima.cpp: Fast 30-70B LLM Inference on Heterogeneous and Low-Resource Home Clusters», ICLR 2026. arxiv.org/abs/2504.08791

[3] exo, documentación y código fuente (descubrimiento, elección y ubicación de capas). github.com/exo-explore/exo

[4] Borzunov et al., «Petals: Collaborative Inference and Fine-tuning of Large Models». arxiv.org/abs/2209.01188

[5] Foro de Ray, soporte de Windows. discuss.ray.io/t/ray-core-x-windows-support/7877

[6] distributed-llama, documentación del proyecto. github.com/b4rtaz/distributed-llama

[7] GPUStack, documentación del proyecto. github.com/gpustack/gpustack

[8] Linux Journal, «Build a Virtual SuperComputer with Process Virtualization» (Incredibuild). linuxjournal.com/node/1338803

[9] Anderson, «BOINC: A Platform for Volunteer Computing». arxiv.org/abs/1903.01699

[10] HTCondor, manual. htcondor.readthedocs.io

[11] Flamenco, guía de inicio. flamenco.blender.org/usage/quickstart

[12] Dask, robo de tareas. distributed.dask.org/en/stable/work-stealing.html

[13] Juice Labs, repositorio comunitario. github.com/Juice-Labs/Juice-Labs

[14] Tailscale, «LM Link», 25/02/2026. tailscale.com/blog/lm-link-remote-llm-access

Fuentes consultadas el 08/10/2026. Las cifras de terceros corresponden a sus equipos de prueba; las de Incredibuild son del fabricante.

Anexo B – Control de versiones

Edición	Fecha	Cambios
1.0	10/2026	Plan inicial de trabajo y validación.
2.0	08/10/2026	Hipótesis con umbrales, variables, investigación tecnológica, diagramas y prototipo v0.1.
3.0	08/10/2026	Formato de documento de proyecto, requerimientos del prototipo y pendientes.
4.0	08/10/2026	Documento único xara_documentacion. Estado del arte con el funcionamiento de cada proyecto, alternativas de construcción, XARA como capa integradora y prototipo v0.2 con registro automático.
4.1	08/10/2026	Instalador de llama.cpp y modelos; P1 registra AVX2 y la velocidad de cada CPU.
4.2	09/10/2026	Capítulo 14 ampliado: usos por tipo de usuario, usos que no sirven, segmentos y modelo gratis/pago.

Anexo C – Archivos del proyecto

Archivo	Contenido
xara/xara_documentacion.docx	Este documento
xara/prototipo/GUIA.md	Guía no técnica: instalación, arranque, uso y funcionamiento

Archivo	Contenido
xara/prototipo/README.md	Referencia técnica: modos, variables, protocolo y pruebas
xara/prototipo/xara.py	Prototipo v0.2 (coordinador, nodo, pruebas y registro)
xara/prototipo/panel.html	Panel web
xara/prototipo/iniciar_xara.bat	Lanzador para Windows
xara/prototipo/abrir_cortafuegos.bat	Apertura de puertos en redes privadas de Windows
xara/prototipo/instalar_ia.bat	Descarga de llama.cpp y de los modelos de prueba
xara/prototipo/prompts.txt	Preguntas de la prueba P4
xara/diagramas/*.puml	Diagramas editables en PlantUML
xara/.generar_xara_documentacion.py	Generador de este documento